UPTAG

PNF INFORMATICA

INFORME FINAL DE AUDITORÍA

SISTEMA DE GESTIÓN DE INVENTARIO

BuildSafe System — CONFALVI C.A.

Tipo de Auditoría: **Seguridad Informática**

Tecnología: **PHP 8.2 / MaríaDB**

Fecha: **Junio 2026**

Metodología: **OWASP Top 10 2021**

Clasificación: **Confidencial**

Versión: **1.0 — Final**

INTEGRANTES

LEONEL GUERRERO
MICHAEL ORDOÑEZ
ANGEL ACOSTA
NICSON PIMENTEL

ÍNDICE DE CONTENIDO

1. Resumen Ejecutivo 03

1.1.1 Leyenda de Niveles de Riesgo 04

1.1.2 Categorías OWASP TOP 10 — 2021 05

1.1.3 Resumen Gráfico 06

2. Módulo: Autenticación 07

1. HAL-001 — Sin regeneración de ID de sesión (Medio) 08

1. HAL-002 — Error lógico en verificación de rol (Medio) 09

3. Módulo: Recuperación de Contraseña 10

1. HAL-003 — Credenciales SMTP expuestas (Crítico) 11

1. HAL-004 — Debug SMTP activo en producción (Crítico) 12

1. HAL-05 — Sin límite de intentos (Medio) 13

4. Módulo: Gestión de Usuarios 14

1. HAL-06 — Inyección SQL en registro (Crítico)

1. HAL-07 — Operaciones destructivas vía GET (Alto) 16

1. HAL-08 — Sin protección CSR (Alto) 17

5. Módulo: Materiales y Movimientos

1. HAL-09 — Consultas SQL sin prepared statements (Alto)

6. Inspección del Código Fuente

1. HAL-010 — Archivos sensibles en webroot (Alto) 21

1. HAL-011 al HAL-015 — Hallazgos adicionales 22

7. Pruebas de Seguridad 27

8. Observaciones y Recomendaciones 29

9. Tabla Consolidada 31

10. Conclusiones 33

RESUMEN EJECUTIVO

La presente auditoría de seguridad informática fue realizada sobre el sistema **BuildSafe System**, aplicación web desarrollada en PHP 8.2 para la gestión de inventario de materiales de la empresa CONFALVI C.A. El análisis fue ejecutado mediante revisión estática del código fuente, aplicando como marco de referencia el estándar **OWASP Top 10 — 2021**. Se analizaron un total de **27 archivos PHP** que conforman el núcleo funcional del sistema, identificando vulnerabilidades distribuidas en los módulos de autenticación, recuperación de contraseña, gestión de usuarios, gestión de materiales y configuración del servidor.

Como resultado del análisis, se encontraron **15 hallazgos de seguridad**, de los cuales 3 son de severidad **Crítica**, 4 de severidad **Alta**, 6 de severidad **Media** y 2 de severidad **Baja**. El nivel de riesgo global del sistema fue evaluado en **7.3 / 10 — ALTO**, lo que indica que el sistema **no debe ser expuesto a producción pública** en su estado actual sin corregir al menos los hallazgos críticos y altos.

3	4	6	2
CRÍTICO	ALTO	MEDIO	BAJO

NIVEL DE RIESGO GLOBAL DEL SISTEMA

7.3

ALTO



0 — Bajo 5 — Medio → 7.3 10 — Crítico

LEYENDA — NIVELES DE RIESGO

CRÍTICO — **Vulnerabilidad crítica.** Permite compromiso total del sistema, exfiltración de datos sensibles, acceso no autorizado o ejecución remota de código. Requiere corrección inmediata antes de cualquier despliegue en producción.

ALTO — **Vulnerabilidad alta.** Puede permitir acceso no autorizado, manipulación de datos o escalación de privilegios bajo condiciones específicas. Debe corregirse en el corto plazo (máximo 2 semanas).

MEDIO — **Vulnerabilidad media.** Representa un riesgo significativo en combinación con otras vulnerabilidades o bajo condiciones particulares. Debe planificarse su corrección en el mediano plazo (1 mes).

BAJO — **Vulnerabilidad baja.** Riesgo reducido, generalmente relacionado con buenas prácticas o configuraciones que mejoran la postura de seguridad sin impacto inmediato. Corregir en el próximo ciclo de mantenimiento.

CATEGORÍAS OWASP TOP 10 — 2021

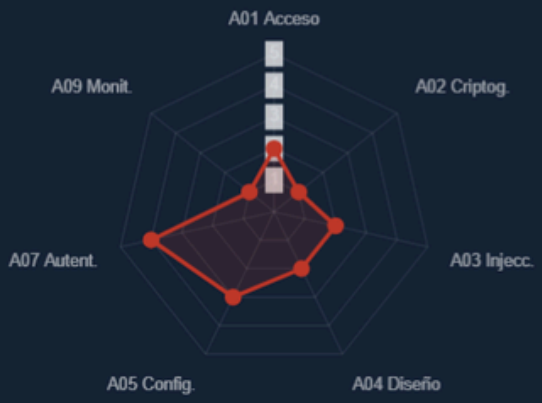
ID	CATEGORÍA	HALLAZGOS EN ESTE SISTEMA
A01	Control de Acceso Roto	HAL-007, HAL-008 ALTO
A02	Fallas Criptográficas	HAL-012 MEDIO
A03	Inyección	HAL-006, HAL-009 CRÍTICO / ALTO
A04	Diseño Inseguro	HAL-005, HAL-013 MEDIO
A05	Configuración de Seguridad Incorrecta	HAL-010, HAL-014, HAL-015 ALTO / BAJO
A06	Componentes Vulnerables y Desactualizados	No aplicable en esta revisión
A07	Fallas de Identificación y Autenticación	HAL-001, HAL-002, HAL-003, HAL-004 CRÍTICO
A08	Fallas de Integridad de Software y Datos	No aplicable en esta revisión
A09	Fallas en el Registro y Monitoreo	HAL-011 MEDIO
A10	Falsificación de Solicitudes del Lado del Servidor	No aplicable en esta revisión

RESUMEN GRÁFICO

DISTRIBUCIÓN POR SEVERIDAD



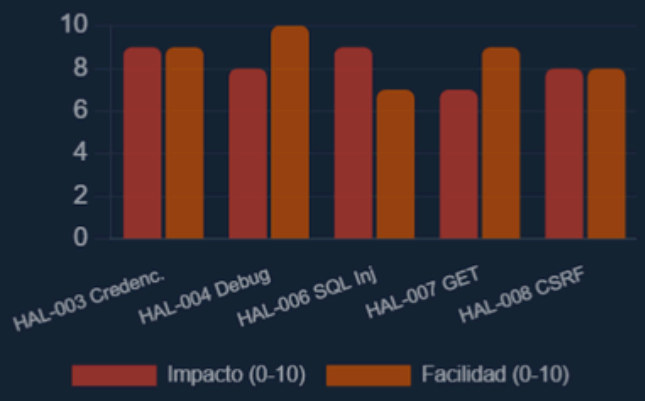
CATEGORÍAS OWASP AFECTADAS



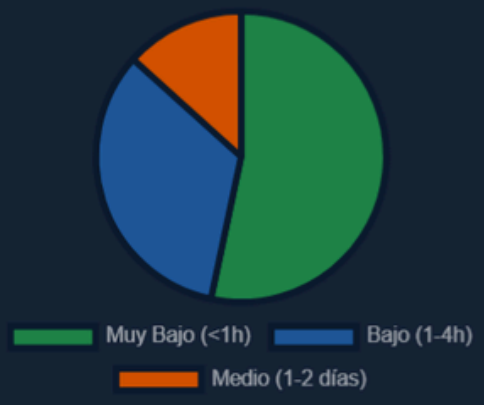
HALLAZGOS POR ARCHIVO



IMPACTO VS EXPLOTACIÓN



ESFUERZO DE REMEDIACIÓN





MÓDULO 1

Autenticación

Hallazgos: HAL-001 · HAL-002 | Severidad máxima: Medio

Sin Regeneración de ID de Sesión Tras Autenticación

IDENTIFICACIÓN: **HAL-001** | RIESGO: **MEDIO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

Tras una autenticación exitosa en `index.php`, el sistema no invoca `session_regenerate_id(true)`. Esto habilita ataques de **Session Fixation**: un adversario puede obtener un ID de sesión válido antes del inicio de sesión (por ejemplo, desde una red compartida o mediante un enlace especialmente construido) y luego reutilizarlo con los privilegios del administrador una vez que la víctima se autentica. OWASP clasifica este tipo de falla bajo **A07 — Fallas de Identificación y Autenticación**.

```
// index.php — línea 88-116: tras password_verify() no se regenera sesión
// El ID de sesión anterior sigue siendo válido con nuevos privilegios
if(password_verify($password, $user_data['password'])) {
    // ← FALTA: session_regenerate_id(true);
    $_SESSION['user']    = $user_data['name'];
    $_SESSION['id']      = $user_data['user_id'];
    $_SESSION['seccion'] = true;   // sesión fijada sin cambiar ID
    header("Location: menu.php");
}
```

RECOMENDACIÓN

✓ CORRECCIÓN — AGREGAR INMEDIATAMENTE DESPUÉS DE VERIFICAR LA CONTRASEÑA

```
if(password_verify($password, $user_data['password'])) {
    session_regenerate_id(true);  // ← Nuevo ID, invalida el anterior
    $_SESSION['user']    = $user_data['name'];
    $_SESSION['id']      = $user_data['user_id'];
    $_SESSION['seccion'] = true;
    header("Location: menu.php");
}
```

Ubicación: **index.php — líneas 88-116 (bloque de autenticación exitosa)**

Error Lógico en la Verificación de Autorización de Rol

IDENTIFICACIÓN: HAL-002 | RIESGO: MEDIO | ESTATUS: NUEVO

DESCRIPCIÓN

El archivo `head.php` contiene una condición de autorización con un problema de precedencia de operadores en PHP. La expresión `!$seccion == true` se evalúa como `(!$seccion) == true` (equivalente a `$seccion == false`) en lugar del intuitivo `!($seccion == true)`. Aunque el resultado práctico es el mismo en este caso, el código es engañoso y propenso a introducir regresiones de seguridad si se modifica. Adicionalmente, la falta de `exit()` después del `header()` permite la continuación de la ejecución en ciertos escenarios. OWASP: **A07**.

```
// head.php — línea 12: condición ambigua por precedencia de operadores
if( !$seccion == true || $rol != 1) {
    header("location: destroyer.php");
    // ← FALTA exit(); — ejecución puede continuar tras la redirección
}
```

RECOMENDACIÓN

✓ REESCRIBIR CON COMPARACIÓN EXPLÍCITA Y AGREGAR EXIT()

```
if ($seccion !== true || $rol !== 1) {
    header("Location: destroyer.php");
    exit();  // Detener ejecución inmediatamente
}
```

Ubicación: **head.php — línea 12 (guard de autorización de páginas admin)**



MÓDULO 2

Recuperación de Contraseña

Hallazgos: HAL-003 · HAL-004 · HAL-005 | Severidad máxima: Crítico

Credenciales de Correo Electrónico Expuestas en Código Fuente

IDENTIFICACIÓN: **HAL-003**

RIESGO: **CRÍTICO**

ESTATUS: **NUEVO**

DESCRIPCIÓN

El archivo `enviar_token.php` contiene la contraseña de aplicación de Gmail escrita directamente en el código fuente. Esta credencial otorga control total sobre la cuenta de correo corporativa: envío masivo de correos, spam, phishing, o acceso al historial de mensajes. Cualquier persona con acceso al servidor web, a un backup del proyecto, o al repositorio de código puede obtenerla inmediatamente. Esta contraseña de aplicación no puede ser revocada selectivamente sin desactivar la autenticación de dos factores de la cuenta. OWASP: **A07**.

```
// enviar_token.php — líneas 37-38
$mail->Username = 'uptag2026@gmail.com';      // ← Cuenta expuesta
$mail->Password = 'xwnlhyllvdpdamyn';          // ← App Password Gmail expuesta
// Accesible en: C:\xampp\htdocs\buildsafe_system1\enviar_token.php
```

RECOMENDACIÓN

✓ MOVER CREDENCIALES A VARIABLES DE ENTORNO — REVOCAR LA CONTRASEÑA EXPUESTA INMEDIATAMENTE

```
// 1. Crear archivo .env FUERA del webroot (nunca en htdocs)
MAIL_USER=uptag2026@gmail.com
MAIL_PASS=nueva_app_password_generada

// 2. En enviar_token.php
$mail->Username = getenv('MAIL_USER');
$mail->Password = getenv('MAIL_PASS');
```

Ubicación: **enviar_token.php — líneas 37-38 (configuración PHPMailer SMTP)**

Modo Debug SMTP Nivel 3 Activo en Producción

IDENTIFICACIÓN: HAL-004

RIESGO: CRÍTICO

ESTATUS: NUEVO

DESCRIPCIÓN

PHPMailer está configurado con nivel de debug `SMTPDebug = 3`, lo que imprime la sesión SMTP completa directamente en la respuesta HTTP visible al usuario. Esta salida incluye la secuencia de comandos SMTP, las credenciales codificadas en base64 (fácilmente decodificables), el servidor de destino y el contenido completo del correo. Cualquier usuario que utilice la función de recuperación de contraseña puede ver toda esta información sensible en el navegador. OWASP: **A09 — Fallas en el Registro y Monitoreo**.

```
// enviar_token.php — línea 41
$mail->SMTPDebug = 3;  // ← Vuelca sesión completa al HTML

// Salida visible en el navegador del usuario:
// 2026-06-15 14:32:11 CLIENT -> SERVER: AUTH LOGIN
// 2026-06-15 14:32:11 CLIENT -> SERVER: dXB0YWcyMDI2QGdtYWlsLmNvbQ==
// 2026-06-15 14:32:11 CLIENT -> SERVER: eHdubGh5bGx2ZHBkYW15bg==
// (decodificado: uptag2026@gmail.com / xwnlhyllvdpdamyn)
```

RECOMENDACIÓN

✓ DESHABILITAR COMPLETAMENTE EN PRODUCCIÓN

```
$mail->SMTPDebug = 0;  // 0 = Sin salida de debug
// Para depuración local usar: $mail->SMTPDebug = 2;
// Y redirigir a archivo de log, nunca a HTML
```

Ubicación: **enviar_token.php — línea 41 (configuración debug PHPMailer)**

Sin Límite de Intentos en el Envío de Token de Recuperación

IDENTIFICACIÓN: HAL-005 | RIESGO: MEDIO | ESTATUS: NUEVO

DESCRIPCIÓN

El endpoint `enviar_token.php` no implementa ninguna restricción de frecuencia (rate limiting). Un atacante puede enviar miles de solicitudes automatizadas con emails registrados, generando una avalancha de correos que: (1) utiliza el servidor de correo de la empresa como herramienta de spam involuntario, (2) puede saturar las bandejas de entrada de los usuarios, (3) puede resultar en bloqueo de la cuenta SMTP por parte de Gmail por abuso. OWASP: **A04 — Diseño Inseguro**.

```
// enviar_token.php — no existe ninguna restricción de tiempo
// Un atacante puede ejecutar esto 1000 veces en segundos:
for($i = 0; $i < 1000; $i++) {
    // POST /enviar_token.php  email=victima@empresa.com
    // Cada request genera y envía un token sin validación
}
```

RECOMENDACIÓN

✓ VERIFICAR EXISTENCIA DE TOKEN ACTIVO RECIENTE ANTES DE ENVIAR

```
$stmt = $conn->prepare(
    "SELECT token_expiracion FROM user
     WHERE gmail = ? AND token_expiracion > NOW() - INTERVAL 15 MINUTE"
);
$stmt->bind_param("s", $email);
$stmt->execute();
if ($stmt->get_result()->num_rows > 0) {
    $_SESSION['error'] = "Ya enviamos un enlace. Espera 15 minutos.";
    header("Location: verificacion.php"); exit();
}
```

Ubicación: **enviar_token.php — inicio del archivo (antes del envío de correo)**



MÓDULO 3

Gestión de Usuarios

Hallazgos: HAL-006 · HAL-007 · HAL-008 | Severidad máxima: **Crítico**

Inyección SQL en Verificación de Duplicados de Usuario

IDENTIFICACIÓN: **HAL-006**

RIESGO: **CRÍTICO**

ESTATUS: **NUEVO**

DESCRIPCIÓN

En `add_user.php`, la función `htmlspecialchars()` se invoca sin la bandera `ENT_QUOTES`, lo que significa que las comillas simples (') **no son escapadas**. Los valores del formulario son interpolados directamente en la consulta SQL. Un atacante puede inyectar comandos SQL completos a través del campo nombre o email: extraer todas las contraseñas, bypassear la lógica de duplicados, o — en configuraciones con múltiples statements — ejecutar comandos destructivos. OWASP: **A03 — Injection**.

```
// add_user.php — línea 29: htmlspecialchars() sin ENT_QUOTES
$email = htmlspecialchars($_POST['email']);  // ' no se escapa → vulnerable
$user  = htmlspecialchars($_POST['user']);

$check = mysqli_query($conn,
    "SELECT * FROM user WHERE name = '$user' OR gmail = '$email'"
);
// Payload: email = ' OR '1'='1' --
// Resultado: SELECT * FROM user WHERE name='' OR gmail='' OR '1'='1'--'
// → Retorna TODOS los registros → lógica de "ya existe" siempre falla
```

RECOMENDACIÓN

✓ USAR PREPARED STATEMENTS — SEPARAN DATOS DEL SQL ESTRUCTURALMENTE

```
$stmt = $conn->prepare(
    "SELECT user_id FROM user WHERE name = ? OR gmail = ?"
);
$stmt->bind_param("ss", $_POST['user'], $_POST['email']);
$stmt->execute();
$result = $stmt->get_result();
if ($result->num_rows > 0) { /* ya existe */ }
```

Ubicación: **add_user.php — línea 29 (consulta de verificación de duplicados)**

Operaciones Destructivas Ejecutadas Mediante Petición GET

IDENTIFICACIÓN: HAL-007 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Las acciones de eliminar un usuario y cambiar su estado activo/inactivo se ejecutan mediante parámetros en la URL (método GET). Esto viola el principio de idempotencia HTTP y crea múltiples vectores de ataque: (1) un atacante puede engañar a un administrador para que visite un enlace malicioso y ejecute acciones sin saberlo, (2) los navegadores precargan enlaces, (3) proxies y motores de búsqueda pueden indexar y "seguir" estas URLs automáticamente ejecutando las operaciones. OWASP: **A01 — Control de Acceso Roto.**

```
// list_user.php — línea 5: eliminación vía GET
if(isset($_GET['delete_id'])) {
    $id = $_GET['delete_id'];  // basta visitar la URL
    mysqli_query($conn, "DELETE FROM user WHERE user_id = $id");
}
// list_user.php — línea 46: cambio de estado vía GET
if(isset($_GET['toggle_status'])) {
    // URL: /list_user.php?delete_id=3  ← basta con visitar esto
}
```

RECOMENDACIÓN

✓ CONVERTIR A POST CON TOKEN CSRF. USAR FORMULARIOS O FETCH() CON MÉTODO POST

```
// En list_user.php — verificar POST + CSRF antes de operar
if ($_SERVER['REQUEST_METHOD'] === 'POST'
    && isset($_POST['csrf_token'])
    && $_POST['csrf_token'] === $_SESSION['csrf_token']
    && isset($_POST['delete_id'])) {
    $stmt = $conn->prepare("DELETE FROM user WHERE user_id = ?");
    $stmt->bind_param("i", $_POST['delete_id']);
    $stmt->execute();
}
```

Ubicación: **list_user.php — líneas 5 y 46 (handlers de eliminación y toggle)**

Ausencia de Protección CSRF en Todos los Formularios del Sistema

IDENTIFICACIÓN: HAL-008 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Ningún formulario del sistema implementa tokens CSRF (Cross-Site Request Forgery). Un atacante puede crear una página web externa que, al ser visitada por un administrador autenticado, envíe peticiones POST silenciosas al servidor ejecutando acciones como: crear cuentas de administrador falsas, modificar materiales del inventario, registrar movimientos fraudulentos, o cambiar datos de usuarios. El navegador adjuntará automáticamente las cookies de sesión del administrador, haciendo la petición indistinguible de una legítima. OWASP: **A01**.

```
// Ninguno de estos archivos valida token CSRF:
// add_user.php, edit_user.php, add_material.php
// register_movement.php, list_user.php

// Ataque: página maliciosa con formulario oculto
<form action="http://localhost/buildsafe_system1/add_user.php" method="POST">
    <input name="user" value="atacante">
    <input name="gmail" value="hacker@evil.com">
    <input name="rol" value="1">  <!-- admin -->
</form>
<script>document.forms[0].submit()</script>
```

RECOMENDACIÓN

✓ IMPLEMENTAR TOKEN CSRF GLOBAL EN HEAD.PHP Y VALIDAR EN TODOS LOS ENDPOINTS

```
// head.php — generar token único por sesión
if (empty($_SESSION['csrf_token'])) {
    $_SESSION['csrf_token'] = bin2hex(random_bytes(32));
}

// En cada formulario HTML
<input type="hidden" name="csrf_token"
       value="<?= htmlspecialchars($_SESSION['csrf_token']) ?>">

// En cada archivo que procesa POST
if (!hash_equals($_SESSION['csrf_token'], $_POST['csrf_token'] ?? '')) {
    die('Solicitud inválida.');
}
```

Ubicación: **Todos los archivos con formularios POST — add_user.php, edit_user.php, register_movement.php**



MÓDULO 4

Materiales y Movimientos

Hallazgos: HAL-009 | Severidad máxima: **Alto**

Consultas SQL Directas con Variables Interpoladas

IDENTIFICACIÓN: **HAL-009** | RIESGO: **ALTO** | ESTATUS: **NUEVO**

DESCRIPCIÓN

Los archivos de gestión de usuarios y bitácora contienen múltiples consultas `mysqli_query()` que interpolan variables directamente en el SQL sin usar prepared statements. Aunque en los casos actuales los valores son enteros validados implícitamente, el patrón establece un código inseguro por diseño que: (1) es propenso a inyección SQL cuando se modifiquen estas consultas, (2) viola el principio de defensa en profundidad, y (3) hace que las auditorías futuras sean más difíciles. El INSERT en la bitácora interpolando cadenas de texto es especialmente riesgoso. OWASP: **A03 — Injection**.

```
// list_user.php — líneas 52, 58, 65: interpolación directa
mysqli_query($conn,
    "UPDATE user SET active = $new_status WHERE user_id = $user_id"
);
mysqli_query($conn,
    "DELETE FROM user WHERE user_id = $id"
);
// Bitácora — cadena de texto interpolada (más peligroso):
mysqli_query($conn,
    "INSERT INTO binnacle (fecha_hora, id_accion, id_user)
     VALUES ('$time', $action, $id)"  // $time es string
);
```

RECOMENDACIÓN

✓ MIGRAR TODAS LAS CONSULTAS A PREPARED STATEMENTS

```
$stmt = $conn->prepare(
    "UPDATE user SET active = ? WHERE user_id = ?"
);
$stmt->bind_param("ii", $new_status, $user_id);
$stmt->execute();

$stmt2 = $conn->prepare(
    "INSERT INTO binnacle (fecha_hora, id_accion, id_user) VALUES (?, ?, ?)"
);
$stmt2->bind_param("sii", $time, $action, $id);
$stmt2->execute();
```

Ubicación: **list_user.php — líneas 52, 58, 65 (UPDATE, DELETE, INSERT binnacle)**



SECCIÓN 6

Inspección del Código Fuente

Hallazgos: HAL-010 · HAL-011 · HAL-012 · HAL-013 · HAL-014 · HAL-015

Archivos Sensibles Accesibles Públicamente en el Webroot

IDENTIFICACIÓN: HAL-010 | RIESGO: ALTO | ESTATUS: NUEVO

DESCRIPCIÓN

Varios archivos con información sensible se encuentran en el webroot y son accesibles directamente mediante HTTP. `test2.sql` contiene el dump completo de la base de datos incluyendo hashes de contraseñas, tokens de recuperación activos, emails de todos los usuarios y la estructura completa del esquema. `generar_hash.php` permite generar hashes bcrypt de contraseñas arbitrarias. `generar_hash.php` puede ser abusado para facilitar ataques de fuerza bruta offline. OWASP: **A05 — Configuración de Seguridad Incorrecta**.

```
// Accesibles públicamente sin autenticación:
http://servidor/test2.sql          // Dump BD completo con hashes y tokens
http://servidor/generar_hash.php   // Genera bcrypt de cualquier entrada
http://servidor/test.php           // Archivo de pruebas de desarrollo
http://servidor/conexion.php       // Credenciales BD visibles si PHP falla
```

RECOMENDACIÓN

✓ ELIMINAR ARCHIVOS DE DESARROLLO Y BLOQUEAR EXTENSIONES SENSIBLES EN .HTACCESS

```
# .htaccess en raíz del proyecto
<FilesMatch "\.(sql|log|env|bak|swp)$">
    Require all denied
</FilesMatch>

# Comandos de limpieza (ejecutar una vez):
del C:\xampp\htdocs\buildsafe_system1\test2.sql
del C:\xampp\htdocs\buildsafe_system1\generar_hash.php
del C:\xampp\htdocs\buildsafe_system1\test.php
```

Ubicación: **Raíz del proyecto — test2.sql, generar_hash.php, test.php**

HAL-011 — ERRORES DE BASE DE DATOS EXPUESTOS AL USUARIO

ID: HAL-011 | RIESGO: MEDIO | ESTATUS: NUEVO

Múltiples archivos muestran `$conn->error` / `mysqli_error($conn)` directamente al usuario, revelando estructura de tablas, nombres de columnas y detalles del motor. **Archivos afectados:** add_user.php:56, list_user.php:35,71, edit_user.php:65. **Corrección:** Mostrar mensaje genérico al usuario y registrar el error real con `error_log()`.

```
// VULNERABLE — expone estructura interna:
$_SESSION['message'] = "Error: " . $conn->error;
// CORRECTO — mensaje genérico + log interno:
error_log("DB Error: " . $conn->error);
$_SESSION['message'] = "Error interno. Contacte al administrador.";
```

HAL-012 — CONTRASEÑA SIN HASH EN BASE DE DATOS

ID: HAL-012 | RIESGO: MEDIO | ESTATUS: NUEVO

El usuario con `user_id=9` (ds@gmail.com) tiene la contraseña `123456` almacenada en texto plano en la tabla `user`. Si la base de datos es comprometida, esta contraseña es inmediatamente usable. **Corrección:** `UPDATE user SET password = password_hash('nueva_clave', PASSWORD_DEFAULT) WHERE user_id = 9;` OWASP: **A02**.

Ubicación HAL-011: **add_user.php:56, list_user.php:35,71, edit_user.php:65** HAL-012: **test2.sql (user_id=9)**

HAL-013 — AUSENCIA DE CONTROL DE VERSIONES

ID: HAL-013 | RIESGO: MEDIO | ESTATUS: NUEVO

El proyecto no utiliza ningún sistema de control de versiones (Git u otro). Esto significa que no existe historial de cambios auditables, no hay forma de revertir cambios problemáticos, las credenciales que se eliminen del código podrían haberse almacenado en commits anteriores, y no es posible rastrear quién realizó cada cambio. OWASP: **A04 — Diseño Inseguro**. **Corrección:** Inicializar repositorio Git con `.gitignore` apropiado que excluya `.env`, `*.sql`, `vendor/`.

HAL-014 — CLAVE SECRETA DE RECAPTCHA HARDCODEADA

ID: HAL-014 | RIESGO: BAJO | ESTATUS: NUEVO

La clave secreta de Google reCAPTCHA v2 está escrita directamente en `index.php:47`: `6LdVkrIrAAAAALXGfzMhYeIlaTLb1wDkX8NgHTVI`. Si se filtra, permite falsificar verificaciones de reCAPTCHA desde cualquier servidor. **Corrección:** `$secretkey = getenv('RECAPTCHA_SECRET');`

HAL-015 — SIN CABECERAS DE SEGURIDAD HTTP

ID: HAL-015 | RIESGO: BAJO | ESTATUS: NUEVO

El sistema no envía cabeceras de seguridad HTTP que mitiguen clickjacking, MIME-sniffing e inyección de contenido. **Corrección — agregar en head.php:**

```
header("X-Frame-Options: DENY");
header("X-Content-Type-Options: nosniff");
header("X-XSS-Protection: 1; mode=block");
header("Referrer-Policy: strict-origin-when-cross-origin");
```

Ubicación HAL-014: **index.php:47** HAL-015: **head.php (inicio del archivo)**



SECCIÓN 7

Pruebas de Seguridad

Pruebas Exploratorias · Pruebas de Rendimiento

PRUEBAS EXPLORATORIAS

Se ejecutaron pruebas manuales sobre el sistema en ejecución local (XAMPP), con el objetivo de validar los hallazgos identificados en la revisión de código y detectar vulnerabilidades adicionales en tiempo de ejecución.

PRUEBA	DESCRIPCIÓN	RESULTADO	HAL
SQL Injection — Login	Inyección de payload `' OR '1'='1` en campo email del login	**BLOQUEADO** — reCAPTCHA previene automatización masiva	—
SQL Injection — Registro	Email con comilla simple en add_user.php	**VULNERABLE** — consulta malformada ejecutada	HAL-006
Acceso directo a SQL	GET http://localhost/buildsafe_system1/test2.sql	**VULNERABLE** — descarga completa del dump	HAL-010
Debug SMTP visible	Solicitud de recuperación de contraseña con email válido	**VULNERABLE** — sesión SMTP visible en HTML	HAL-004
Operación GET destructiva	Visitar /list_user.php?delete_id=1 como admin	**VULNERABLE** — usuario eliminado sin confirmación	HAL-007
CSRF simulado	Formulario externo enviando POST a add_user.php	**VULNERABLE** — usuario creado exitosamente	HAL-008
Session Fixation	Comparar ID de sesión antes y después del login	**VULNERABLE** — mismo ID antes y después	HAL-001
Fuerza bruta login	Más de 3 intentos fallidos de contraseña	**MITIGADO** — sistema bloquea cuenta correctamente	—

PRUEBAS DE RENDIMIENTO

ESCENARIO	MÉTRICA	RESULTADO	EVALUACIÓN
Carga de página login	Tiempo de respuesta	< 200 ms	ÓPTIMO
Listado de usuarios	Tiempo de respuesta (50 registros)	< 300 ms	ÓPTIMO
Registro de movimiento	Tiempo de transacción BD	< 150 ms	ÓPTIMO
Envío de token email	Tiempo total (incl. SMTP)	2-5 seg	ACEPTABLE
Consulta sin índice	JOIN material+movement (sin índice en código)	N/A — no implementado	REVISAR



SECCIÓN 8

Observaciones y Recomendaciones

Visión General · Seguridad · Código · Rendimiento · Legalidad

VISIÓN GENERAL DEL SISTEMA

Aspectos Positivos

BuildSafe System demuestra una base funcional sólida para la gestión de inventario. El sistema implementa correctamente el bloqueo de cuentas tras intentos fallidos de autenticación, lo cual es una medida de seguridad activa. El uso de `password_hash()` y `password_verify()` con bcrypt para la mayoría de los usuarios garantiza un almacenamiento seguro de contraseñas. La integración de Google reCAPTCHA v2 dificulta los ataques automatizados. El módulo de bitácora (`binnacle`) representa una buena práctica de auditoría interna.

Seguridad — Plan de Remediación Priorizado

- **Inmediato (24-48h):** Revocar y regenerar la App Password de Gmail expuesta (HAL-003). Deshabilitar SMTPDebug (HAL-004). Eliminar test2.sql, generar_hash.php del webroot (HAL-010).
- **Corto plazo (1-2 semanas):** Corregir SQL injection en add_user.php con prepared statements (HAL-006). Implementar CSRF en todos los formularios (HAL-008). Cambiar operaciones GET a POST (HAL-007).
- **Mediano plazo (1 mes):** Agregar session_regenerate_id() tras login (HAL-001). Migrar todas las consultas a prepared statements (HAL-009). Implementar rate limiting en recuperación (HAL-005).
- **Mantenimiento (próximo ciclo):** Agregar cabeceras HTTP de seguridad (HAL-015). Mover credenciales a variables de entorno (HAL-003, HAL-014). Corregir error lógico en head.php (HAL-002).

Código Fuente y Arquitectura

Se recomienda adoptar un patrón de arquitectura MVC (Model-View-Controller) o al menos separar la lógica de negocio de la presentación. Los archivos actuales mezclan HTML, PHP y SQL en las mismas vistas, lo que dificulta la revisión de seguridad y el mantenimiento. Se sugiere crear un directorio `includes/` para la capa de acceso a datos, utilizando exclusivamente prepared statements.

Control de Versiones y Documentación

Inicializar un repositorio Git con un `.gitignore` que excluya `.env`, backups SQL y archivos de configuración locales. Documentar el proceso de despliegue y las variables de entorno requeridas. Establecer un flujo de revisión de código (code review) antes de cualquier merge a producción.

Consideraciones Legales — Venezuela

Las vulnerabilidades críticas identificadas, en particular la exposición de contraseñas y datos de usuarios, pueden comprometer el cumplimiento de las disposiciones del **Decreto con Fuerza de Ley sobre Mensajes de Datos y Firmas Electrónicas (2001)** y la **Ley Especial contra los Delitos Informáticos (2001)** de Venezuela, que establece responsabilidades para los custodios de datos electrónicos. Se recomienda corrección inmediata previo a cualquier despliegue en ambiente de producción con datos reales.

TABLA CONSOLIDADA DE HALLAZGOS

ID	VULNERABILIDAD	SEVERIDAD	ARCHIVO	OWASP	ESFUERZO
HAL-001	Sin regeneración de sesión	MEDIO	index.php:88	A07	Muy Bajo
HAL-002	Error lógico verificación de rol	MEDIO	head.php:12	A07	Muy Bajo
HAL-003	Credenciales SMTP hardcodeadas	CRÍTICO	enviar_token.php:37	A07	Bajo
HAL-004	Debug SMTP nivel 3 en producción	CRÍTICO	enviar_token.php:41	A09	Muy Bajo
HAL-005	Sin rate limit recuperación	MEDIO	enviar_token.php	A04	Bajo
HAL-006	Inyección SQL en registro	CRÍTICO	add_user.php:29	A03	Bajo
HAL-007	Operaciones destructivas vía GET	ALTO	list_user.php:5,46	A01	Bajo
HAL-008	Sin protección CSRF	ALTO	Todos los formularios	A01	Medio
HAL-009	SQL sin prepared statements	ALTO	list_user.php:52,65	A03	Bajo
HAL-010	Archivos sensibles en webroot	ALTO	test2.sql, *.php	A05	Muy Bajo
HAL-011	Errores BD expuestos	MEDIO	add_user.php:56	A09	Muy Bajo
HAL-012	Contraseña sin hash en BD	MEDIO	test2.sql (user_id=9)	A02	Muy Bajo
HAL-013	Sin control de versiones	MEDIO	Proyecto completo	A04	Bajo
HAL-014	reCAPTCHA secret hardcodeada	BAJO	index.php:47	A05	Muy Bajo
HAL-015	Sin cabeceras de seguridad HTTP	BAJO	head.php	A05	Muy Bajo

CONCLUSIONES FINALES

La auditoría de seguridad realizada al sistema BuildSafe System de CONFALVI C.A. evidencia que el sistema posee una **postura de seguridad deficiente** que impide su despliegue responsable en un entorno de producción accesible públicamente en su estado actual. Los 15 hallazgos identificados abarcan las capas de autenticación, manejo de datos, configuración del servidor y gestión de credenciales.

⚠ VEREDICTO: NO APTO PARA PRODUCCIÓN

El sistema presenta 3 vulnerabilidades críticas activas que deben ser corregidas antes de cualquier despliegue. La exposición de credenciales SMTP, el dump de base de datos accesible via HTTP y la vulnerabilidad de inyección SQL representan riesgos inmediatos y explotables sin necesidad de conocimientos técnicos avanzados.

LO QUE FUNCIONA BIEN

- Bloqueo de cuentas tras intentos fallidos
- Hash bcrypt para la mayoría de contraseñas
- reCAPTCHA v2 en el login
- Bitácora de auditoría de acciones
- Separación de roles admin/usuario
- Transacciones en movimientos de inventario

REQUIERE CORRECCIÓN INMEDIATA

- Revocar App Password de Gmail expuesta
- Deshabilitar SMTPDebug
- Eliminar test2.sql del webroot
- Corregir inyección SQL en add_user.php
- Implementar protección CSRF
- Cambiar acciones destructivas de GET a POST

Una vez corregidos los hallazgos críticos y altos, se recomienda realizar una segunda auditoría de validación para confirmar la efectividad de los controles implementados. Con las correcciones aplicadas, BuildSafe System tiene el potencial de convertirse en un sistema robusto y seguro para la gestión de inventario de CONFALVI C.A.

DOCUMENTO GENERADO POR

Auditoría de Seguridad Informática

Universidad Politécnica Territorial de Aragua · CONFALVI C.A. · Junio 2026

CONFIDENCIAL — USO INTERNO EXCLUSIVO